```
import asyncio
import json
import logging
import os
from datetime import time
from pathlib import Path
from zoneinfo import ZoneInfo

import pandas as pd
import yfinance as yf
from telegram import Update
from telegram.ext import (Application, CommandHandler, ContextTypes,
                          MessageHandler, filters)

logging.basicConfig(level=logging.INFO)
TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
WATCH_FILE = Path("watchlists.json")
watchlists: dict[str, list[str]] = (
    json.loads(WATCH_FILE.read_text()) if WATCH_FILE.exists() else {}
)


def save_watchlists():
    WATCH_FILE.write_text(json.dumps(watchlists))


# ---------- المؤشرات الفنية ----------
def rsi(close: pd.Series, n: int = 14) -> pd.Series:
    d = close.diff()
    gain = d.clip(lower=0).ewm(alpha=1 / n, adjust=False).mean()
    loss = (-d.clip(upper=0)).ewm(alpha=1 / n, adjust=False).mean()
    return 100 - 100 / (1 + gain / loss)


def macd(close: pd.Series):
    line = close.ewm(span=12, adjust=False).mean() - close.ewm(span=26, adjust=Fa
    signal = line.ewm(span=9, adjust=False).mean()
    return line, signal


def analyze(ticker: str) -> dict | None:
    df = yf.Ticker(ticker).history(period="1y")
    if df.empty or len(df) < 30:
        return None
```

```
    c = df["Close"]
    m, s = macd(c)
    r = rsi(c)
    out = {
        "ticker": ticker.upper(),
        "price": c.iloc[-1],
        "change": (c.iloc[-1] / c.iloc[-2] - 1) * 100,
        "rsi": r.iloc[-1],
        "macd_up": m.iloc[-1] > s.iloc[-1],
        "macd_cross": (m.iloc[-1] > s.iloc[-1]) != (m.iloc[-2] > s.iloc[-2]),
        "sma50": c.rolling(50).mean().iloc[-1],
        "sma200": c.rolling(200).mean().iloc[-1],
        "high52": df["High"].max(),
        "low52": df["Low"].min(),
    }
    return out


def fmt(a: dict) -> str:
    notes = []
    if a["rsi"] >= 70:
        notes.append("🔴 RSI في منطقة تشبّع شراء")
    elif a["rsi"] <= 30:
        notes.append("🟢 RSI في منطقة تشبّع بيع")
    else:
        notes.append("⚪ RSI محايد")
    notes.append("🟢 MACD فوق خط الإشارة" if a["macd_up"] else "🔴 MACD خط الإشارة
    if a["macd_cross"]:
        notes.append("⚡ حدث تقاطع MACD اليوم")
    if pd.notna(a["sma50"]):
        notes.append("🟢 السعر فوق SMA50" if a["price"] > a["sma50"] else "🔴 تحـ
    if pd.notna(a["sma200"]):
        notes.append("🟢 السعر فوق SMA200" if a["price"] > a["sma200"] else "🔴 ت
    arrow = "📈" if a["change"] >= 0 else "📉"
    return (
        f"{arrow} {a['ticker']}: ${a['price']:.2f} ({a['change']:+.2f}%)\n"
        f"RSI: {a['rsi']:.1f}\n"
        f"أعلى/أدنى 52 أسبوع: ${a['high52']:.2f} / ${a['low52']:.2f}\n\n"
        + "\n".join(notes)
        + "\n\n⚠️ استثمارية. وليس توصية فقط، للمعلومات."
    )


# ---------- الأوامر ----------
HELP = (
    "أرسل رمز سهم أمريكي مثل AAPL لتحليله.\n\n"
    "/watch TSLA أضف سهم لقائمتك -\n"
```

```
    "/unwatch TSLA احذفه -\n"
    "/list عرض القائمة -\n"
    "/report تحليل كل قائمتك الآن -\n\n"
    "أرسل لك يوميًا بعد إغلاق السوق تنبيهًا إذا حصل شيء مهم في أسهمك."
)


async def start(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(HELP)


async def reply_analysis(update: Update, ticker: str):
    a = await asyncio.to_thread(analyze, ticker)
    await update.message.reply_text(fmt(a) if a else f"ما لقيت بيانات للرمز {tick


async def on_text(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    await reply_analysis(update, update.message.text.strip().upper())


async def watch(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    if not ctx.args:
        return await update.message.reply_text("مثال: /watch AAPL")
    t = ctx.args[0].upper()
    lst = watchlists.setdefault(str(update.effective_chat.id), [])
    if t not in lst:
        lst.append(t)
        save_watchlists()
    await update.message.reply_text(f"تمت إضافة {t} ✅")


async def unwatch(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    if not ctx.args:
        return await update.message.reply_text("مثال: /unwatch AAPL")
    t = ctx.args[0].upper()
    lst = watchlists.get(str(update.effective_chat.id), [])
    if t in lst:
        lst.remove(t)
        save_watchlists()
    await update.message.reply_text(f"تم حذف {t}")


async def list_cmd(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    lst = watchlists.get(str(update.effective_chat.id), [])
    await update.message.reply_text("قائمتك: " + ", ".join(lst) if lst else "اضية
```

```
async def report(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    lst = watchlists.get(str(update.effective_chat.id), [])
    if not lst:
        return await update.message.reply_text("استخدم /watch لإضافة أسهم إلى القائمة.")
    for t in lst:
        await reply_analysis(update, t)


# ---------- التنبيه اليومي ----------
async def daily_alerts(ctx: ContextTypes.DEFAULT_TYPE):
    for chat_id, tickers in watchlists.items():
        for t in tickers:
            a = await asyncio.to_thread(analyze, t)
            if a and (a["rsi"] >= 70 or a["rsi"] <= 30 or a["macd_cross"]):
                await ctx.bot.send_message(chat_id, "🔔 تنبيه\n" + fmt(a))


def main():
    app = Application.builder().token(TOKEN).build()
    app.add_handler(CommandHandler(["start", "help"], start))
    app.add_handler(CommandHandler("watch", watch))
    app.add_handler(CommandHandler("unwatch", unwatch))
    app.add_handler(CommandHandler("list", list_cmd))
    app.add_handler(CommandHandler("report", report))
    app.add_handler(MessageHandler(filters.Regex(r"^[A-Za-z.\-]{1,6}$"), on_text))
    # بعد إغلاق السوق الأمريكي (4:30 مساءً بتوقيت نيويورك)، أيام الأسبوع فقط
    app.job_queue.run_daily(
        daily_alerts,
        time(16, 30, tzinfo=ZoneInfo("America/New_York")),
        days=(0, 1, 2, 3, 4),
    )
    app.run_polling()


if __name__ == "__main__":
    main()
```